UNIVERSITY OF ASIA PACIFIC
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
Third Year, Second Semester

Course Name: Software Engineering Lab
Course Code: CSE 314
Date of Submission: 06 - MAR -2026

Submitted by:	**Submitted to:**
MD Sajjad-Bin-Kashem - 23101206	**Tanjina Helaly**
Anamika Rani Dhar - 23101221	**Assistant Professor**
Arnob Saha Bosu Jit -23101222	**Department of CSE,UAP**
Ashpaqur Rahman -23101225	

SOFTWARE DESIGN SPECIFICATION (SDS)

Evalify

Measure Learning. Improve Outcomes

OBE-Based Smart Learning Management & Assessment System

Prepared By: Anamika Rani Dhar , Md Sajjad-Bin-Kashem. And Arnob Saha Bosu Jit .

Group Members Introduction:

1. Md Sajjad-Bin-Kashem-23101206

Role : System Analyst/Business Analyst and System Designer/Software Architect

2. Anamika Rani Dhar - 23101221

Role : Tester/Quality Assurance (QA) Engineer

3. Arnob Saha Bosu Jit - 23101222

Role : Software Developer (Programmer)

4. Ashpaqur Rahman - 23101225

Role : Project Manager

1. Introduction

1.1 Purpose

This document specifies the software requirements to provide a comprehensive and definitive technical blueprint for the development of **Evalify**, an integrated, Outcome-Based Education (OBE) smart learning management and assessment system. It serves as a formal agreement between the development team and stakeholders to ensure that all functional and non-functional needs are successfully met.

The core objectives of the system are:

- **Bridging the Integration Gap:** Eliminating the disconnect between traditional Learning Management Systems (LMS) and OBE assessment processes.
- **Enhancing Academic Accuracy:** Automating complex calculations for Course Learning Outcomes (CLOs) and Program Learning Outcomes (PLOs) to minimize human error and reduce faculty workload.
- **Centralized Academic Resource Hub:** Empowering students by providing a dedicated, efficient repository for course materials, including standardized lecture notes and archives of previous semester question papers.
- **Maintaining Academic Integrity:** Providing built-in tools for plagiarism detection.
- **Facilitating Data-Driven Decisions:**Providing faculty and students with real-time performance analytics to track institutional compliance and student progress.

It is designed to modernize academic administration by replacing manual, fragmented workflows with a centralized, intelligent platform , also to be a student-centric hub that ensures transparency in learning outcomes, secures academic data, and provides the essential tools needed for effective and efficient academic success.

1.2 Scope

The scope of the Evalify system focuses on bridging the critical fragmentation between traditional Learning Management Systems and modern Outcome-Based Education frameworks.Evalify is specifically engineered to support the **"Academic Quality Loop"**.

The system encompasses the following core domains:

- **OBE Mapping:** Enables the structured mapping of assignments to CLOs, PLOs, and Bloom's Taxonomy levels.

- **Automated Assessment:** Replaces manual mark distribution with automated attainment calculations, which reduces faculty workload and minimizes human error.
- **Academic Integrity:** Features built-in plagiarism detection to ensure the authenticity of student submissions.
- **Performance Analytics:** Provides real-time performance dashboards and attainment reports for students and faculty.
- **Resource Management:** Maintains a centralized repository of organized digital notes and previous semester practice questions to improve learning efficiency.

1.3 Intended Audience

The Evalify system is designed for the following stakeholder groups:

- **Development Team:** To serve as the primary technical blueprint for implementing system architecture, database schemas, and integration logic.
- **Academic Administrators (UAP):** To utilize the reporting engine for institutional-level PLO attainment summaries and quality assurance compliance.
- **Faculty Members:** To manage course creation, define OBE mapping rules, utilize digital rubrics for grading, and monitor real-time class attainment data
- **Students:** To utilize the system as a "Progress Navigator" to track individual CLO/PLO attainment, identify weak learning areas, and access centralized academic resources (notes/previous question papers)

Intended Use

Evalify is a specialized, smart Outcome-Based Education (OBE) management system designed to modernize academic administration. Its intended use cases include:

- Centralized OBE Workflow
- Automated Assessment
- Academic Integrity
- Academic Resource Hub
- Real-Time Analytics

1.4 References

- IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications.

- "Evalify" Project Proposal: The initial feasibility study and project justification report.
- Software Requirements Specification Of Evalify.

2.Design Overview

2.1 Design Goals

The design of Evalify aims to establish a robust, automated, and student-centric platform tailored for OBE within the university academic environment. The system follows a structured modular architecture, ensuring that core OBE evaluation processes, plagiarism detection, and resource management are tightly integrated yet logically distinct.

The following design goals guide the overall system architecture and implementation:

Scalability

Even though the system utilizes a centralized architecture, it is designed to support increasing demands from:

- Users (students, faculty, and administrators)
- Course-mapping configurations
- Assignment submissions
- Performance data analytics
- High volumes of simultaneous OBE attainment calculations
- Peak-load access during assessment and grading periods
- Growing historical academic data Scalability is achieved through
- Efficient database indexing for CLO/PLO mapping
- Optimized backend query design for real-time analytics
- Modular scaling of the assessment and reporting engines

Maintainability

Maintainability ensures that the system can be updated to comply with changing accreditation standards (e.g., BAETE) without major restructuring. To achieve this:

- The application code is organized into structured Django apps.
- Business rules for OBE mapping are clearly separated from the user interface.

- Code is properly documented for future institutional handovers.
- Reusable modules are implemented for grading rubrics and plagiarism checks. The internal code organization ensures clarity and allows for seamless feature updates.

Security

Security is a critical design objective to protect sensitive student performance and grading data. The system ensures:

- Secure password hashing and encrypted session management.
- ORM-based database interaction to prevent SQL injection.
- Protection against CSRF attacks.
- Strict Role-Based Access Control (RBAC) to ensure grading privacy.
- Encrypted storage for academic records and assessment data.

These mechanisms ensure that only authorized faculty can modify results, maintaining complete academic integrity.

Performance

The system is designed to provide immediate feedback to both faculty and students under academic load. Performance goals include:

- Rapid OBE attainment calculation upon mark entry.
- Low-latency loading for the "Progress Navigator" dashboards.
- Efficient processing of plagiarism similarity analysis.
- Minimal response time for report generation. Efficient backend logic and structured execution pipelines ensure smooth performance, minimizing the administrative burden on faculty.

Reliability

The system must remain stable and available during critical academic periods, such as final grading cycles. Reliability is ensured through:

- Automated database backup strategies (every 24 hours).
- Robust exception handling during assessment calculations.
- Stable containerized deployment configurations.
- Zero-data-loss policies for academic attainment records.

The system is designed to ensure continuous availability, keeping performance data accessible whenever needed.

Usability

The system provides a clean, intuitive, and responsive interface for:

- Students
- Faculty
- Administrators The interface ensures:
- A responsive design for access on any device.
- Clear visualization of CLO/PLO attainment.
- Simple navigation for accessing course materials, lecture notes, and previous semester question papers.
- Intuitive "Progress Navigator" dashboards.

A user-friendly interface improves student engagement, simplifies complex OBE workflows, and ensures transparency in the grading process.

By prioritizing scalability to handle concurrent academic workloads, maintainability to adapt to evolving accreditation requirements, and security to safeguard intellectual property and sensitive data, the system provides a robust foundation for institutional growth.

Furthermore, the focus on high performance and reliability ensures that both faculty and students experience a seamless workflow, free from the bottlenecks of manual, fragmented assessment processes. Through a user-friendly, mobile-first design, Evalify not only meets the technical demands of a modern OBE framework but also empowers students with the academic insights and resources necessary for success.

2.2 Design Principles

Although the Evalify system is designed to be a streamlined, integrated OBE platform, it adheres to rigorous software engineering principles to ensure sustainability, accuracy, and compliance with accreditation standards.

Logical Separation of Concerns

Even within a unified platform, the system logic is strictly partitioned to prevent architectural drift:

- User Interface handling : Separation of student/faculty dashboards.
- OBE Mapping Logic: Dedicated processing of CLO/PLO attainment algorithms.

- Database Management: Centralized management of academic attainment and student records.
- Assessment & Plagiarism Engine: Isolated processing for content similarity and grade calculation.

Internal Modularity

The system is divided into highly specialized internal modules that operate independently at the code level:

I. Auth & RBAC Module: Manages user roles and secure access.
II. OBE Mapping Module: Handles Bloom's taxonomy and CLO/PLO weightage configurations.
III. Assessment & Rubric Module: Manages automated grading and rubric-based evaluations.
IV. Integrity & Plagiarism Module: Handles similarity analysis for student submissions.
V. Resource & Analytics Module: Manages the "Progress Navigator" and centralized study material repository.

Reusability

To maintain consistency and reduce development overhead, the system utilizes reusable components:

- Standardized Rubric Templates: Applicable across multiple courses.
- OBE Calculation Logic: Shared formulas for attainment tracking.
- Common UI Components: Reusable grid layouts for dashboard widgets.
- Data Validation Middleware: Uniform checks for grading and submission inputs.

High Cohesion

Each module performs a singular, focused responsibility, ensuring that the system is easy to maintain and test:

- OBE Engine: Dedicated solely to calculating attainment based on pre-defined mapping.

- Plagiarism Suite: Handles only submission similarity and authenticity checks.
- Resource Manager: Manages file storage and retrieval for study materials.
- Dashboard Engine: Dedicated purely to data visualization and analytics.

High cohesion ensures that updates to the grading logic do not inadvertently impact the plagiarism detection or resource management modules.

5. Low Coupling

Modules interact through well-defined internal interfaces, ensuring:

- Easier Testing: Modules can be tested in isolation to verify OBE accuracy.
- Fewer Side Effects: Changes to the UI do not impact the core calculation engine.
- Simplified Debugging: Problems can be isolated to a specific domain .

6. Secure-by-Design

Security is embedded into the foundation of Evalify to ensure that sensitive academic data remains private and compliant with institutional quality standards. Implemented features include:

- Role-Based Authorization (RBAC): Restricting access to sensitive attainment data.
- Secure Password Storage: Utilizing robust hashing protocols.
- Input Validation: Preventing data corruption in mark entry fields.
- Encrypted Data Handling: Protecting academic records both at rest and in transit.
- Audit Logs: Tracking changes to mapping and grading to maintain transparency.

This "secure-by-design" approach minimizes vulnerabilities and ensures that the platform is ready for official academic audits.

2.3 System Context

The System Context defines the boundary of the Evalify system. As an integrated OBE-based smart learning management and assessment platform, Evalify acts as a centralized digital ecosystem. While it operates as a cohesive unit, it manages distinct

workflows for assessment, OBE mapping, and content delivery within a single unified deployment environment.

2.3.1 External Entities

Students

Students interact with the system via web browser to:

- View CLO/PLO attainment dashboards .
- Access centralized lecture notes and previous semester question papers.
- Submit assignments and view rubric-based grades.
- Receive performance feedback and identify learning gaps.

Faculty Members

Faculty members access privileged dashboards to:

- Configure OBE mapping rules (CLO-PLO weighting).
- Create assignments and define grading rubrics.
- Manage and upload learning resources.
- Monitor real-time attainment statistics and class performance.
- Analyze student integrity reports.

Administrator

The administrator:

- Manages user roles and departmental access.
- Monitors system logs and performance metrics.
- Maintains database integrity and OBE mapping consistency.
- Handles periodic automated backups.
- Ensures institutional compliance and system stability.

External Plagiarism Detection

The system communicates with external services to:

- Scan student submissions for plagiarism.
- Analyze similarity content.
- Return authenticity reports to maintain academic integrity.

OBE Assessment Engine

The core engine is an integrated module within Evalify that:

- Processes grade inputs against defined rubrics.
- Calculates attainment levels for specific CLOs and PLOs.
- Maps individual assignment results to broader curriculum outcomes.
- Generates audit-ready reports for accreditation purposes.

MySQL Database

The database acts as the central repository for the system, storing:

- User profiles and credentials.
- CLO/PLO mapping data.
- Assignment submissions and grades.
- Lecture notes and archived exam papers. It is directly connected to the Django application through ORM.

Grading & OBE Flow:

User (Faculty) → Assignment Module → OBE Engine → Mapping Calculation → Database → User Interface (Student/Faculty Dashboard)

Resource Flow:

User (Student) → Resource Repository → Retrieval Module → File Storage → User Interface

3. System Architecture

3.1 Architectural Overview

The Evalify system is built upon a Model-View-Controller (MVC) Architecture. This design pattern ensures a clean separation between the user interface, the complex OBE business logic, and the data storage. By merging the organizational benefits of MVC with the structural integrity of a Layered Architecture, the system remains maintainable, testable, and audit-ready.

MVC Architecture

To achieve architectural excellence, we define the roles of these patterns independently before integration:

MVC Architecture:

In the Evalify system, the MVC pattern is the core organizational framework. By decoupling the data logic, the presentation interface, and the request orchestration, we ensure that the complex OBE attainment calculations remain distinct from the user interface.

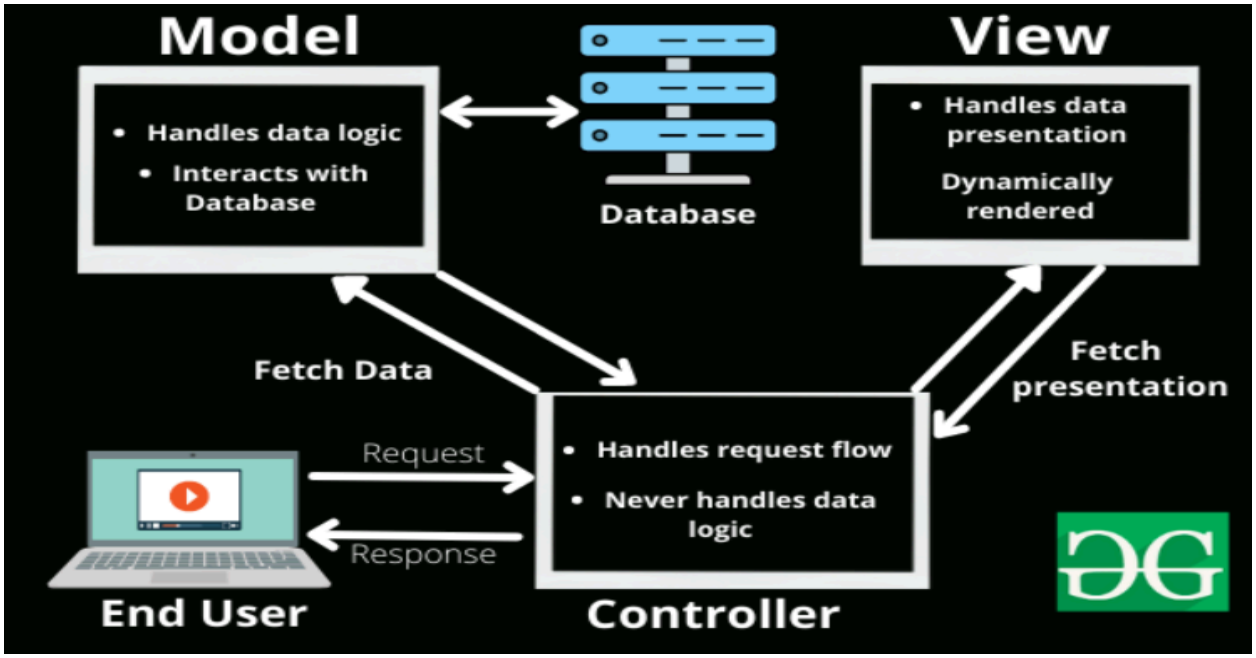


Fig 1: MVC Architecture Design

➢ **Model**

The Model layer serves as the system's single source of managing the database schema and encapsulating the core domain entities of the OBE framework.

Student Model:

- Stores core profile information, enrollment status, and academic program identifiers.
- Maintains a direct link to the Attainment Model, which tracks real-time progress across various courses.

Faculty Model:

- Contains privileged access data, department associations, and curriculum ownership flags.
- Links to the Assignment Model to track

Plagiarism and integrity Model:

- Maintains historical integrity records for students, acting as a safeguard.

Course & Mapping Models:

- Maintain the relationships between CLOs and PLOs, including the weightage factors required for accreditation.

Assessment and Rubric Models:

- Define the structure of assignments, exam questions, and the grading rubrics used for automated evaluation.

Attainment Models:

- Store the historical and current performance data, acting as the foundation for the "Progress Navigator" analytics.

➢ **Views (Presentation Layer)**

The View layer is responsible for rendering the information in a human-readable format. It is strictly limited to display logic and contains no business processing.

Student Dashboard

- **CLO/PLO Attainment Visualization:** Features interactive charts that visualize real-time mastery of learning outcomes.
- **Assignment/Submission Hub:** Provides a unified portal for submitting work, viewing rubrics before submission, and receiving annotated feedback from faculty.
- **Resource Repository:** A structured view of lecture notes, archived exam papers, and supplementary materials, organized by course and module.
- **Integrity Dashboard:** Displays status indicators on whether submitted work has passed the plagiarism screening process.

Faculty Evaluation Panel

- **Rubric-Based Grading Interface:** Allows faculty to grade individual segments of an assignment against pre-defined OBE criteria.

- **Integrity Verification View:** A side-by-side comparison screen that displays student submissions alongside plagiarism similarity reports, allowing faculty to make informed decisions on authenticity.
- **OBE Analytics Console:** Provides class-wide performance metrics, enabling faculty to identify which CLOs/PLOs the class as a whole is struggling with.
- **Course Management Interface:** Tools to define mapping ratios, update syllabus structures, and schedule assessment cycles.

Administrative Audit View

- **Institutional Reporting Dashboard:** A high-level, aggregated view of attainment trends across departments.
- **System Integrity Logs:** A monitoring view that tracks plagiarism alerts, system usage statistics, and user role management to ensure institutional security.

➢ Controllers

The Controller serves as the central orchestration layer, acting as the intermediary between the user interface and the database.

- **Assessment & OBE Controller:** Manages assignment creation and the attachment of specific rubrics.
- **Integrity & Plagiarism Controller:** Manages student file uploads and coordinates with external APIs to initiate plagiarism.
- **Progress & Analytics Controller:** It processes this information into structured data for the "Progress Navigator".
- **Course & Resource Controller:** Handles the management of digital lecture materials, curriculum outlines, and the archival/retrieval of previous semester question papers.
- **User & Access Controller:** Handles authentication, session management, and Role-Based Access Control (RBAC). It ensures that only authorized faculty can view attainment metrics and that students only access their personal progress data.

MVC	Evalify Functions Handled
View (Templates)	Student Dashboards, Grading UI, Analytics Reports, Resource Browsers.
Controller (Services)	OBE Calculations (CLO/PLO), Plagiarism Checking, Role/Access (RBAC), Grade Processing.
Model (ORM)	User/Student/Faculty Data, Course/Mapping Data, Assessment/Rubric Records, Plagiarism Logs.

3.2 Architectural Diagram

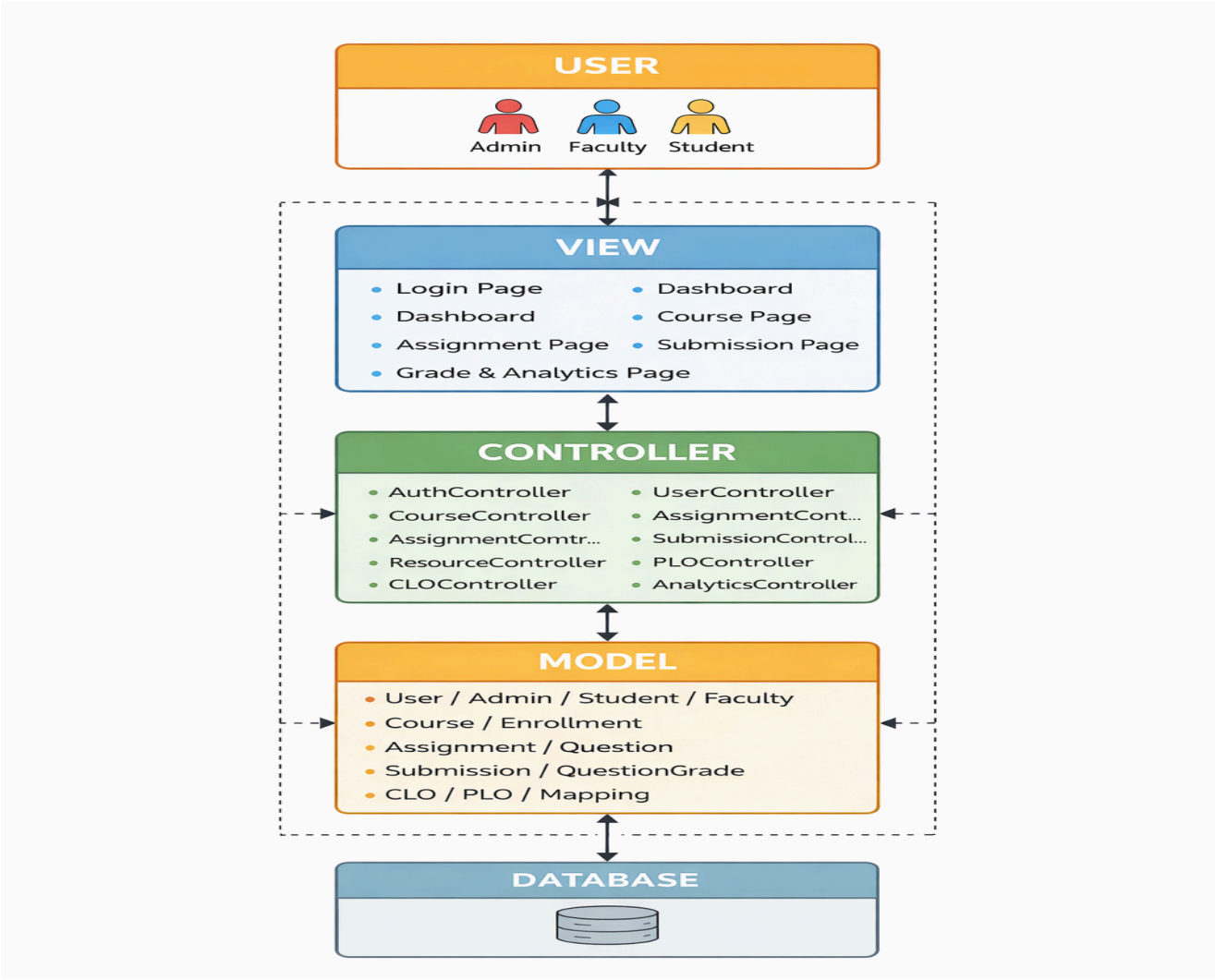


Fig 2: Architectural Diagram of Evalify – OBE Smart LMS

This diagram illustrates the MVC architectural pattern as applied to the Evalify. It organizes the system into three interconnected layers to separate the user interface, business logic, and data management.

1. User Layer

The system is accessed by three distinct user roles, each with different permissions and requirements:

- **Admin:**

  Likely manages system configurations and user roles.

- **Faculty:**

  Handles course content, assignments, and assessment mapping.

- **Student:**

  Interacts with course materials, submits assignments, and views grades.

2. View Layer

This is the **Frontend**—what the users see and interact with. It consists of various pages that request data from the controller and display it to the user:

- **Navigation & Access:**

  Login and Dashboard.

- **Academic Content:**

  Course, Assignment, and Submission pages.

- **Reporting:**

  A dedicated Grade & Analytics page for performance tracking.

3. Controller Layer

The Controller acts as the "brain" or middleman. It processes user input from the View, interacts with the Model, and returns the appropriate response. Key controllers include:

- **Authentication & Users:**

  AuthController and UserController.

- **Academic Management:**

  CourseController, AssignmentController, and SubmissionController.

- **OBE Specifics:**

  CLOController (Course Learning Outcomes) and PLOController (Program Learning Outcomes), which are critical for an Outcome-Based Education system.

4. Model Layer

The Model represents the Backend data structures and business rules. It defines how data is organized before it is stored:

- **Entities:**

  Users, Courses, and Enrollments.

- **Assessment Data:**

  Assignments, Questions, Submissions, and QuestionGrades.

- **OBE Mapping:**

  CLO/PLO Mapping, which links student performance to specific learning goals.

5. Database Layer

At the base of the architecture is the physical **Database**. The Model layer communicates with this persistent storage to save, retrieve, and update all system information.

3.3 Technology Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, Bootstrap 5,JS	Responsive UI and dashboard rendering.
Backend	Python , Django	Core logic, MVC orchestration, and security.

Database	MySQL 8.0	Relational storage for student/PLO/CLO mappings.
Authentication	Django Auth System	Secure login, role-based access
Testing & Debugging	Selenium	Integration testing
Version Control	Git and GitHub	Collaborative development and tracking.

3.4 Deployment Architecture

The combination of Agile methodology and Layered MVC architecture enables Evalify to maintain high academic integrity, scalability, and institutional reliability. This synergy ensures that our OBE (Outcome-Based Education) framework remains both developer-friendly and audit-ready.

This structure ensures:

- Audit-Ready Maintainability: Clear separation between the OBE business logic (calculations) and the presentation layer (UI) makes it simple for developers.
- Enhanced Collaboration and Security: Using Git for version control allows faculty and developers to track every change to the grading engine, ensuring complete accountability.
- Rapid, Reliable Deployment: We can deploy updates, such as new rubric mappings or performance analytics, with minimal downtime, ensuring the platform remains live for students and faculty.
- Isolated Testing:Faster development cycles with continuous testing and deployment opportunities.

4. Data Design

4.1 Data Model Overview

The Evalify system utilizes a Relational Database Management System (RDBMS) via MySQL. The Django ORM (Object-Relational Mapping) acts as the abstraction layer.

The Django ORM ensures high data integrity by managing:

- **Table Creation:** Automatically syncing Python classes with database schemas.
- **Foreign Key Relationships:** Defining strict links between students, assessments, and learning outcomes.
- **Data Validation:** Ensuring that marks and mapping weights conform to defined business rules.

Major Entities

The system is built upon a modular set of entities that work together to track OBE attainment:

Identity & Access Domain- These entities handle who the user is and what they are allowed to do.

- User
- Faculty
- Student

Academic & OBE Framework-

- Course
- Enrollment
- CLO
- PLO
- CLO_PLO_Mapping
- Academic_Resource

Assessment & Evaluation Domain

- Assignment
- Question
- Submission.
- Grade

4.2.Methodology

4.2.1 System Modeling and Architecture:

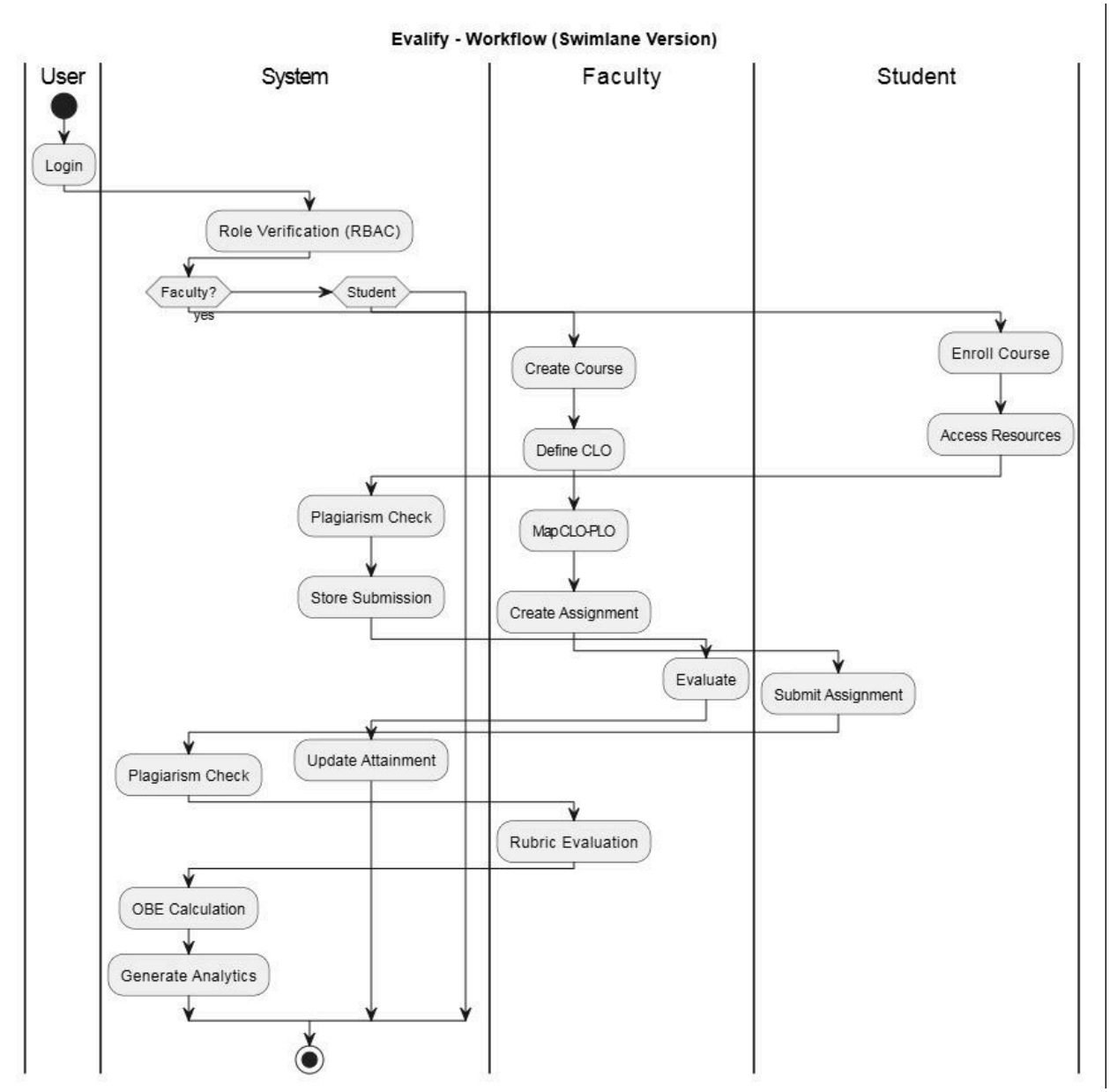


Fig 3

Drive link :https://drive.google.com/drive/u/0/folders/1KssKzsGlAymGDFCWy620PXTIi2cPahx-

The workflow shown in the figure represents the complete operational process of the Evalify system, illustrating how different actors (User, Faculty, Student) interact with the system through structured processing steps. The architecture follows a role-based execution model integrated with OBE automation.

Authentication & Role Verification Process

The workflow begins when a user initiates the login process.

- The user provides credentials (email and password).
- The system performs Role-Based Access Control (RBAC) verification.
- Based on role identification (Faculty or Student), the system redirects the user to their respective functional workflow.

If authentication fails, access is denied.If successful, the user is granted secure session access to the system.

This ensures secure identity verification and controlled access to system modules.

Faculty Operational Workflow

Once verified as Faculty, the workflow proceeds as follows:

Course & OBE Setup

1. Create Course
2. Define Course Learning Outcomes (CLO)
3. Map CLO to PLO

This stage establishes the academic framework required for outcome-based evaluation.

Assignment Creation

- Faculty creates assignments linked to specific CLOs.
- Assignments become available for student submission.

Evaluation & Attainment Processing

After students submit assignments:

- The system performs Plagiarism Check.
- Submissions are stored in the database.
- The faculty performs evaluation using rubric-based grading.
- The system updates attainment records.
- OBE calculation engine computes CLO–PLO attainment.
- Analytics are generated automatically.

This structured process ensures academic integrity and automated outcome tracking.

Student Operational Workflow

If the authenticated user is identified as Student, the workflow proceeds as follows:

1. Enroll in Course
2. Access Learning Resources
3. Submit Assignment

After submission:

- The system stores the submission.
- Plagiarism check is executed.
- Faculty evaluates the submission.
- Students receive feedback and updated attainment status.

This enables transparent tracking of learning progress.

System-Level Processing

The central system performs several automated backend operations:

- Plagiarism verification
- Submission storage
- Attainment update
- OBE calculation
- Performance analytics generation

These operations occur without direct user intervention and ensure consistent academic evaluation.

Use Case Diagram:

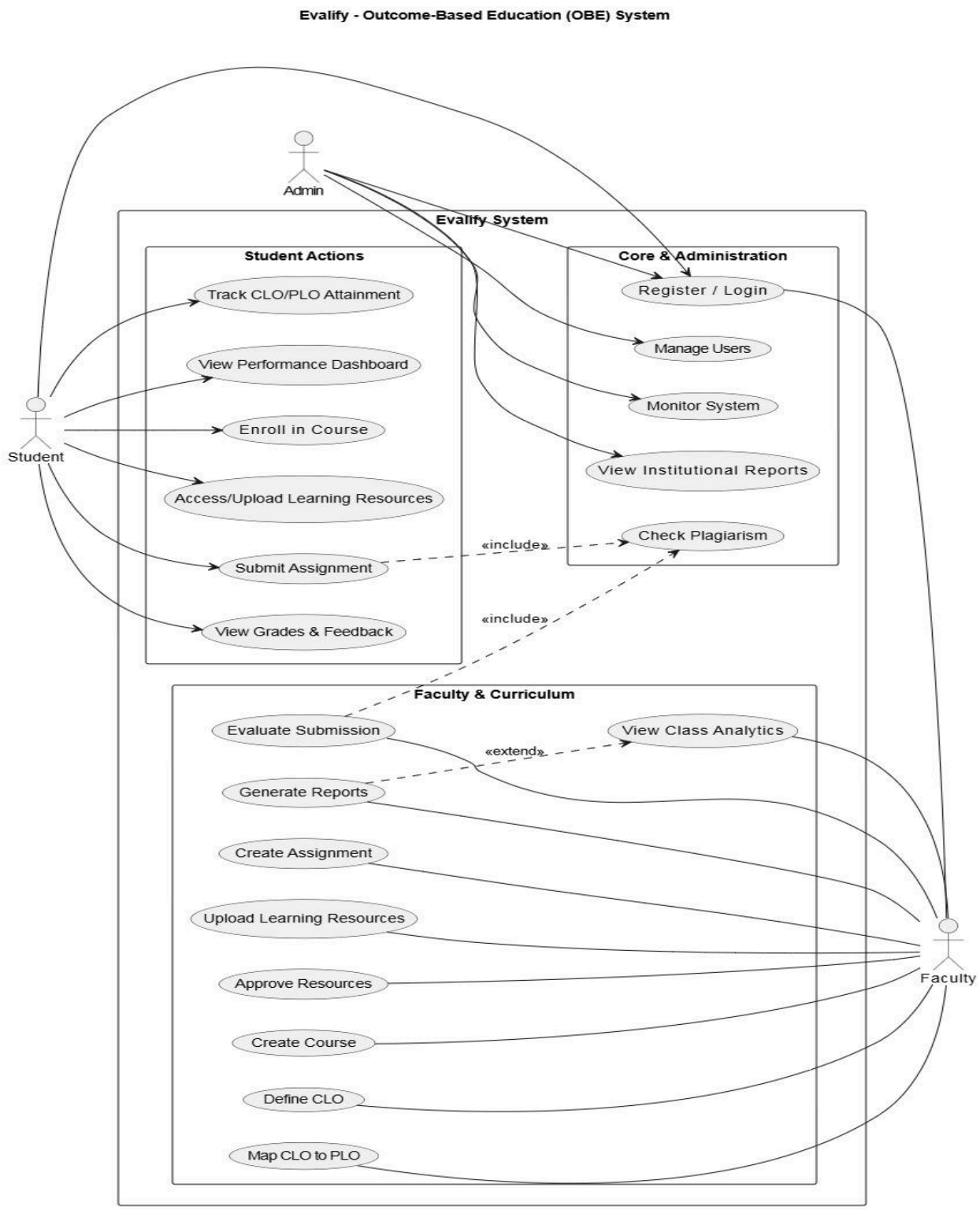


Fig 4: https://drive.google.com/drive/u/0/folders/1KssKzsGlAymGDFCWy620PXTIi2cPahx-

The Use Case Diagram of Evalify represents the functional behavior of the system by identifying the actors and the operations they can perform. It provides a high-level overview of how Students, Faculty, and Admin interact with the platform and defines the system boundary.

The large rectangle represents the Evalify System boundary. All use cases inside it are system functionalities.

Actors in the System

The system has three main actors:

- Student
- Faculty
- Admin

Each actor interacts with specific modules based on their roles and permissions.

i. Student Actor

The Student represents learners enrolled in academic courses.

Main Activities:

- Register / Login
- Enroll in Course
- Access Learning Resources
- Submit Assignment
- View Grades & Feedback
- Track CLO/PLO Attainment
- View Performance Dashboard

Students can monitor their academic progress through attainment tracking and analytics dashboards. When submitting assignments, the system automatically includes plagiarism checking as part of the process.

ii. Faculty Actor

The Faculty represents course instructors responsible for curriculum and assessment management.

Main Activities:

- Create Course
- Define CLO (Course Learning Outcome)
- Map CLO to PLO
- Create Assignment
- Upload Learning Resources
- Approve Resources
- Evaluate Submission
- View Class Analytics
- Generate Reports

Faculty members configure the academic structure of the system. During evaluation, plagiarism checking is included automatically to ensure academic integrity.

iii. Admin Actor

The Admin represents the system administrator responsible for overall platform management.

Main Activities:

- Register / Login
- Manage Users
- Monitor System
- View Institutional Reports

Admin ensures smooth system operation and institutional-level oversight.

Core & Administration Module

This module includes:

- Register / Login
- Manage Users
- Monitor System
- View Institutional Reports
- Check Plagiarism

The “Check Plagiarism” use case is included within submission and evaluation processes, meaning it is automatically executed when required.

Include & Extend Relationships

Include Relationship:

- Submit Assignment → includes → Check Plagiarism
- Evaluate Submission → includes → Check Plagiarism

This means plagiarism verification is mandatory during submission and evaluation.

Extend Relationship:

- Generate Reports → extends → View Class Analytics

This means report generation is an optional extension of analytics.

System Functional Groups

The diagram organizes use cases into three major functional areas:

1. Student Actions
2. Faculty & Curriculum Management
3. Core & Administration

This grouping improves clarity and modular understanding of the system.

ER Diagram:

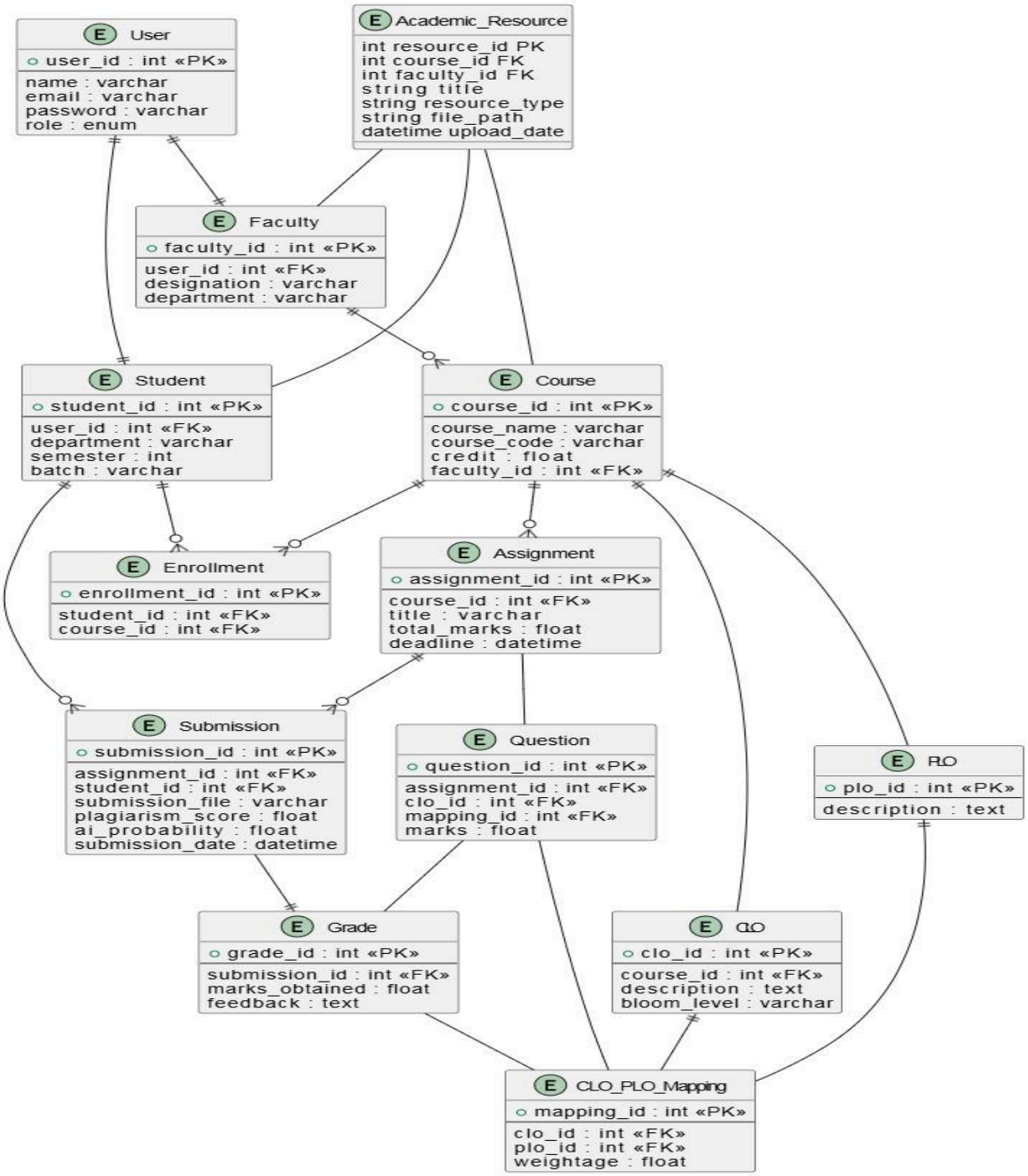


Fig 5: https://drive.google.com/drive/u/0/folders/1KssKzsGlAymGDFCWy620PXTIi2cPahx-

1. Identity & Access Part

The ER Diagram begins with the User entity, which stores authentication-related information such as name, email, password, and role. Based on the role attribute, a user is categorized as either a Student or Faculty. The Student and Faculty entities are linked to the User entity through foreign keys, ensuring proper role-based identity management within the system.

2. Academic Structure Part

The Course entity represents academic subjects created by Faculty members. Each course is associated with a specific faculty and can have multiple students enrolled through the Enrollment entity. The Enrollment table manages the many-to-many relationship between Student and Course, ensuring structured course registration and participation.

3. Assessment & Evaluation Part

Each Course contains multiple Assignments. Assignments are divided into Questions, and each question is mapped to a specific CLO. Students submit their work through the Submission entity, where submission details, plagiarism scores, and AI probability values are stored. After evaluation, results are recorded in the Grade entity, which stores marks and feedback for each submission.

4. OBE Framework Part

The Outcome-Based Education structure is implemented through CLO (Course Learning Outcome) and PLO (Program Learning Outcome) entities. The CLO_PLO_Mapping entity connects CLOs to PLOs with defined weightage values. This mapping enables automated calculation of attainment levels, supporting real-time OBE performance tracking.

5. Resource Management Part

The Academic_Resource entity stores learning materials uploaded by Faculty for specific courses. These resources are linked to both Course and Faculty entities, ensuring proper ownership and structured access to study materials.

6. Overall System Integrity

The ER Diagram maintains referential integrity through primary and foreign keys. It ensures organized data storage, supports automated OBE calculations, enables structured assessment processing, and maintains transparency in academic evaluation within the Evalify system.

DFD (Level 0):

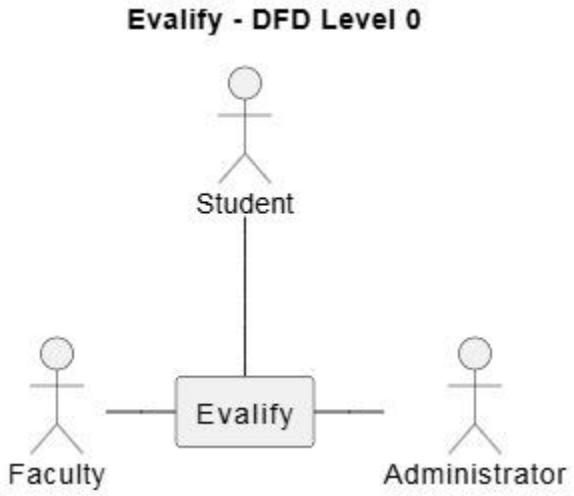


Fig 6

The DFD Level 0 represents the high-level overview of the Evalify system. In this diagram, the entire system is shown as a single central process named “Evalify.” It illustrates how external entities interact with the system without showing internal processes.

There are three main external entities: Student, Faculty, and Administrator.

The Student interacts with the system to enroll in courses, submit assignments, access resources, and view performance results. The Faculty interacts with the system to create courses, define CLOs, manage assignments, and evaluate submissions. The Administrator manages users, monitors system activities, and views institutional reports.

All data flows between these external actors and the Evalify system pass through the central process. The internal operations such as OBE calculation, plagiarism detection, and attainment processing are not shown at this level.

The Context Diagram defines the system boundary and provides a clear understanding of how different users interact with the Evalify platform at a high level.

DFD (Level 1)

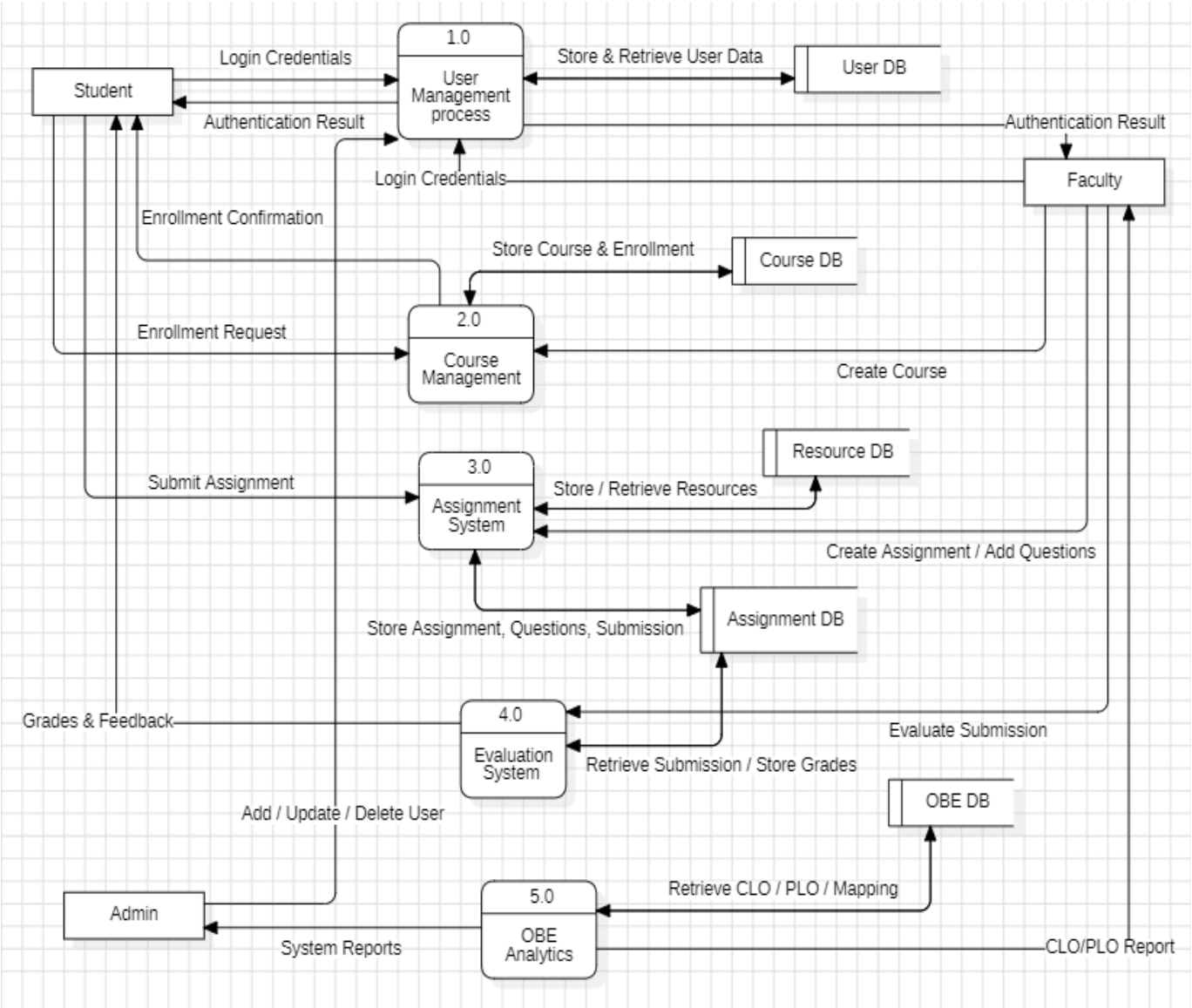


Fig 7: https://drive.google.com/drive/u/0/folders/1KssKzsGlAymGDFCWy620PXTIi2cPahx-

- **1.0 User Management Process**

  This is the gateway for authentication. It receives Login Credentials from Students and Faculty, verifies them against the User DB, and returns an Authentication Result.

- **2.0 Course Management:**

  Faculty initiate this by sending Create Course data. Students send Enrollment Requests, and the process stores these relationships in the Course DB while providing an Enrollment Confirmation back to the Student.

- **3.0 Assignment System:**

  This process handles the creation of academic tasks. Faculty Create Assignments and Add Questions, while Students Submit Assignments. Data is stored in and retrieved from the Resource DB and Assignment DB.

- **4.0 Evaluation System:**

  Faculty use this to Evaluate Submissions. The process retrieves student work from the Assignment DB, stores the resulting marks in the same database, and sends Grades & Feedback directly to the Student.

- **5.0 OBE Analytics:**

  This smart process retrieves mapping data from the OBE DB (containing CLO/PLO/Mapping info). It generates CLO/PLO Reports for Faculty and broader System Reports for the Admin.

Data Storage

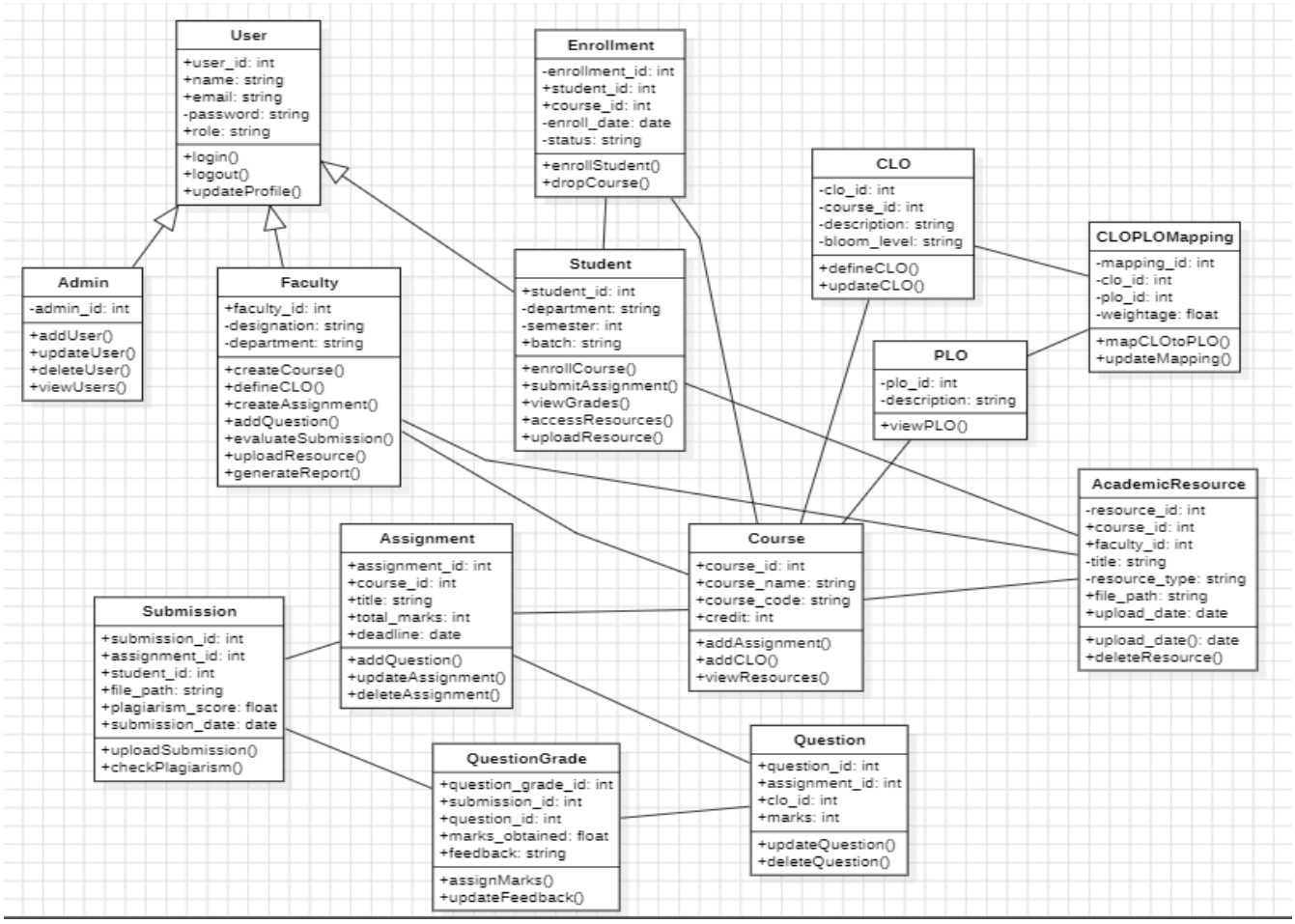
The DFD identifies five distinct data repositories used to maintain system persistence:

- User DB: Stores and retrieves profiles and credentials.
- Course DB: Persists course details and student enrollment records.
- Resource DB: Acts as a repository for academic materials.
- Assignment DB: Stores assignment parameters, specific questions, and student submissions.
- OBE DB: Centralizes the mapping between Course Learning Outcomes and Program Learning Outcomes.

Administrative Control

The Admin entity interacts primarily with the management and reporting ends of the flow. They can Add, Update, or Delete Users through the User Management process and receive final System Reports from the OBE Analytics process.

Class Diagram



1. Outcome-Based Logic (OBE)

The central value of the project is its ability to track learning goals beyond simple grades:

- CLO & PLO Management: Faculty can define Course Learning Outcomes (CLO) and map them to Program Learning Outcomes (PLO) with specific weightages.
- Automated Calculation: The system performs OBE Calculations based on student performance to generate attainment analytics.
- Question-Level Mapping: Assessment is granular; individual Questions are linked to specific CLOs within the system's data model.

2. User Roles & Workflow

The system employs Role-Based Access Control (RBAC) to manage three primary user types:

- Faculty: Responsible for course creation, defining learning outcomes, mapping CLOs to PLOs, creating assignments, and performing Rubric Evaluations.
- Student: Engages in course enrollment, accessing academic resources, and submitting assignments.
- Admin: Manages the broader user base, including adding or deleting users and overseeing system-wide roles.

3. Academic Management Features

- Assessment Lifecycle: Includes assignment creation, student submission, and a built-in Plagiarism Check handled by the system.
- Resource Distribution: Supports the uploading and management of Academic Resources tied to specific courses.
- Grading & Feedback: Faculty can assign marks and provide feedback at the question level through the QuestionGrade entity.

4. Technical Architecture

The project is built on an MVC (Model-View-Controller) pattern to separate concerns:

- Model: Defined by a comprehensive class structure including User, Course, Enrollment, Assignment, and CLOPLOMapping.
- System/Controller: Automates background tasks like Role Verification, Plagiarism Detection, and Analytics Generation.
- View: Provides specific dashboards and pages for students and faculty to interact with their respective academic data.

4.3 Software Process Model:

We have used the Agile Methodology for developing the Evalify system, as it supports iterative development, continuous feedback, and high-level flexibility. Agile allowed us to refine the system through frequent engagement with faculty and students, ensuring that core modules, such as OBE mapping, attainment tracking, and plagiarism detection evolved to meet real-world academic demands.

We followed the Scrum framework, dividing development into manageable sprints that focused on building, testing, and improving features incrementally. This approach resulted in a highly adaptive cycle, allowing us to seamlessly integrate complex OBE calculation engines and dynamic reporting dashboards without compromising system stability, ultimately ensuring that Evalify remains a robust, user-centric platform.

5. Component Design

5.1 Component Overview

Module	Primary Responsibility	Key Entities Involved
Authentication & Profile	Manages user login, roles (RBAC), and user-specific profiles for students and faculty.	User, Faculty, Student
Academic Administration	Manages the institutional hierarchy, including course offerings, student enrollment, and academic terms.	Course , Enrollment
Assessment & Grading	Creation of assignments, rubric-based evaluation, and submission tracking.	Assignment, Rubric, Submission, Grade, Question
OBE Calculation Engine	Processes curriculum mappings and calculates student attainment levels against specific outcomes.	CLO, RO (PLO), CLO_PLO_Mapping, AttainmentRecord
Integrity & Compliance	Monitors academic honesty by tracking similarity scores and generating audit reports for submissions.	PlagiarismRecord
Resource Management	Provides a repository for faculty to upload and share learning materials linked to courses.	Academic_Resource

5.2 Detailed Component Design

Component Name: Authentication & Profile module

Purpose

- Identity Verification
- Access Control (RBAC)
- Data Security
- Session Management

Inputs

- User credentials (username/password)
- registration forms
- profile update submissions

Outputs

- Verified session tokens
- user dashboard data
- authorization status (allowed/denied)

Dependencies

- Database System
- Framework Middleware
- Security Libraries

Internal Logic:

- Secure Authentication (Login Process)
- Validate input fields.
- Hash password.
- Role-Based Access Control (RBAC) Verification
- Generate session token on successful login.
- Session Lifecycle Management

Component Name: Academic Administration module

Purpose

- Manage course offerings
- Facilitating student enrollment
- Provide a centralized repository for academic resources.

Inputs

- Course metadata (course name, code, credits)
- Student enrollment requests
- Batch/semester identifiers
- Faculty-uploaded learning resources.

Outputs

- Validated database records
- Structured class rosters
- Accessible course material repositories

Dependencies

- Course
- Enrollment
- Academic_Resource
- Faculty

Internal Logic

- Course Initialization
- Enrollment Logic
- Resource Management

Component Name: Assessment & Grading module

Purpose

- Student evaluation
- Publication of an assignment
- Final faculty grading
- Bridge raw student work with institutional outcome metrics.

Inputs

- Assignment prompts
- Deadline configurations

- Submission files
- Faculty input for Grade records.

Outputs

- Submission records
- Grade entries
- Feedback text provided to students.

Dependencies

- Assignment
- Question
- Submission
- Grade entities.

Internal Logic

- Validates that submissions occur before the deadline
- Links the marks_obtained to the submission_id.
- Logic aggregates marks at the Question level to support granular OBE mapping.

Component Name: OBE Calculation Engine

Purpose

- Mathematically processes
- Ensuring accreditation compliance

Inputs:

- Raw marks_obtained from the Grading component
- CLO and PLO definitions
- Weightage values from the CLO_PLO_Mapping table.

Outputs

- Aggregated attainment scores for specific CLOs and PLOs
- Updated AttainmentRecord entries.

Dependencies:

- CLO, RO, CLO_PLO_Mapping
- Question
- Grade entities.

Internal Logic:

- The engine identifies the clo_id associated with a Question via the mapping table.
- Calculates the weighted average of student performance effectively mapping.
- Performance to Program Learning Outcomes (PLO)

Component Name: Integrity & Compliance module

Purpose

- Protects the academic integrity of the
- Verify originality of student submissions
- Maintains audit logs.

Inputs

- Student Submission file paths
- External similarity API results.

Outputs

- Plagiarism_score and ai_probability metrics stored in the Submission entity
- Generates audit reports.

Dependencies

- Submission
- PlagiarismRecord entities.

Internal Logic

- Triggers an asynchronous scan of the file content
- It saves the results into the Submission entity attributes
- High-risk scores can trigger an automated flag for faculty review.

Component Name: Resource Management Module

Purpose

- Centralizes the distribution of learning materials
- Ensures that all students have equal access to curriculum-aligned resources.

Inputs:

- Faculty-provided files
- Resource titles
- Resource types (PDF, Slide, Video)
- Course associations.

Outputs

- Organized repository accessible by students
- Persistent Academic_Resource records.

Dependencies

- Academic_Resource.
- Course, and Faculty entities.

Internal Logic

- The module maps resources to a course_id.
- It restricts resource visibility so that students with an Enrollment record for that specific course .

6. Interface Design

6.1 User Interface Design

The User Interface (UI) design for Evalify is focused on reducing the cognitive load on faculty while managing complex OBE data. The interface serves as the primary gateway for students to interact with the Assessment, OBE Engine, and Resource Management modules.

6.1.1 Wireframes

- **Student Dashboard:** A simplified view showing enrolled courses, upcoming assignment deadlines, and a visual that displays real-time CLO/PLO attainment.

- **Faculty Grading Console:** Educators can view student submissions. It features an integrated pane for plagiarism scores, allowing faculty to verify integrity before finalizing grades.

- **Curriculum Mapping Interface:** An admin/faculty view used to establish the CLO_PLO_Mapping. This screen utilizes link Question entities to CLOs and PLOs visually.

6.1.2 UI Standards

Evalify adopts a clean academic design language to maintain professional standards and system usability:

- **Color Palette:**
  - ➔ Professional
  - ➔ high-contrast colors to minimize eye strain
  - ➔ Success (Green) and Alert (Red/Yellow) colors are reserved strictly for grade status
  - ➔ plagiarism indicators.
- **Component Library:**
  - ➔ Standardized components
  - ➔ Progress bars
  - ➔ Consistent modal dialogs for file uploads

The UI is built using a mobile-first approach, ensuring that students can view grades and faculty can check submission statuses on tablets and laptops, maintaining 100% functionality regardless of the device screen size.

6.1.3 Accessibility Considerations

To ensure the platform is inclusive for all users, Evalify follows WCAG 2.1 (Web Content Accessibility Guidelines) standards:

- **Screen Reader Compatibility:** All data visualizations are accompanied by HTML descriptions and tables, allowing screen readers to communicate performance data to visually impaired users.

- **Keyboard Navigation:** Every action is accessible via keyboard shortcuts and focus indicators, ensuring the system does not rely exclusively on mouse input.

- **Contrast & Legibility:** Minimum contrast ratios are enforced for all text elements, and font sizes are adjustable to accommodate diverse visual needs within the institutional environment.

6.2 External Interfaces

6.2.1 Third-Party APIs

- Plagiarism Detection
- Notifications: Email/SMS API
- Cloud Storage

6.2.2 External Systems

- Identity Provider.
- Student Information

6.3 Hardware Interfaces

- The system will be hosted on the university server initially and can be migrated to a cloud-based VPS to ensure scalability as user traffic grows.
- Must support concurrent submissions

6.4 Communication Interfaces

1. Communication Protocols:
    a. HTTPS
    b. TCP/IP
    c. WebSockets
2. Message Formats:
    a. JSON (JavaScript Object Notation).
3. External Service Integration
4. Synchronization Mechanisms

7. Security Design

7.1 Authentication & Authorization

The Evalify system maintains rigorous security standards to protect sensitive academic data, grading records, and plagiarism reports.

Django Authentication Framework

- Evalify utilizes the native Django authentication framework to handle the complete user lifecycle, including registration, authentication, and password management.
- Institutional Verification: Ensuring that only users with verified institutional email addresses (@uap-bd.edu) can create accounts.
- Identity Security: All user passwords are processed using industry-standard, secure hashing algorithms before storage in the database. The system performs strict credential validation during login .

Role-Based Access Control (RBAC)

The system enforces a granular RBAC model to ensure that users interact only with the modules appropriate to their responsibilities:

- Student
- Faculty
- Administrator

Secure Session Handling

Upon successful login,.

- Token Security: Sessions are maintained via secure, HTTP-only cookies that prevent client-side script access,
- Session Lifecycle: The system enforces automatic session timeouts after periods of inactivity, protecting data on shared or public laboratory terminals.

CSRF

- CSRF Protection: Every state-changing request (such as uploading a Submission or updating a Grade) requires a unique, server-validated CSRF token.
- Header Security: The system implements standard HTTP security headers to protect the application .

7.2 Data Security

The Evalify system treats data security as a core architectural requirement. By implementing multi-layered encryption strategies, the system ensures the confidentiality and integrity of student grades, plagiarism reports, and curriculum mappings stored within the database.

Password Hashing

User passwords in Evalify are never stored in plain text.The system utilizes Django's built-in functions to apply strong hashing algorithms (such as PBKDF2 with a SHA256 salt) before storing credentials in the database. During the authentication process, the system validates credentials by comparing the hashed version of the entered password against the stored hash.

Database Constraints

- **Unique Constraints:** Applied to institutional email addresses and Student/Faculty IDs to prevent duplicate accounts and identity conflicts.
- **Referential Integrity:** Foreign key relationships are strictly enforced between User, Course, Enrollment, CLO,PLO , CLO_PLO_MAPPINGS and Submission entities.
- **Integrity Rules:** Cascade deletion rules for example, ensuring that a Grade cannot exist without a corresponding Submission.

Secure File Handling & Isolation

- **Storage Isolation:** Student submissions and faculty resources are stored in isolated directories with restricted execution permissions.
- **Input Validation:** Every file upload is subjected to strict extension validation checking to ensure only authorized document types are processed.

7.3 Security Threat Analysis

Threats	Impact	Mitigation Strategy
Authentication	Unauthorized access to faculty or admin accounts.	Implementation of account lockout policies after multiple failed attempts and mandatory strong password complexity rules.
Injection	Attackers could bypass login or extract sensitive student records from the database.	Use of Django's Object-Relational Mapper.
Cross-Site Scripting	Malicious scripts could be executed in a user's browser to steal session cookies.	Implementation of Django's auto-escaping templates and strict Content Security Policy (CSP) .
Integrity	Compromises the validity of OBE attainment and academic merit.	Integration of the Integrity & Compliance module .
Availability	System downtime during critical assignment deadlines or final grading periods.	Server-side resource monitoring to prevent memory exhaustion.

8. Performance Design

8.1 Performance Requirements

Metric	Target Requirement	Description
Response Time (Standard)	< 3 Seconds	Maximum delay for standard UI interactions, such as course pages or loading profile data.

Response Time (Complex)	< 5 Seconds	Maximum delay for tasks like including generating OBE attainment reports or processing large plagiarism scans.
Concurrent Users	500+ Users	The capacity to support hundreds of active students and faculty members simultaneously.
File Upload Speed	> 2 MB/s	Minimum data transfer rate for uploading academic resources and student submissions to cloud storage.
System Availability	99.9% Uptime	Ensuring the platform is accessible.

9. Error Handling and Logging

9.1 Error Handling Strategy

To ensure Evalify remains reliable during critical academic periods, the system follows a structured approach to managing failures:

Exception Handling

- Errors in specific areas (like the OBE Calculation Engine) are caught locally to prevent the entire system from crashing.
- User-Friendly Feedback
- All exceptions are automatically sent to a central logger for administrators to review and fix without disrupting the user experience.

Retry Mechanisms

- If a database transaction fails due to high traffic, the system re-attempts the operation .
- The interface allows users to manually retry interrupted file uploads.

10. Design Constraints

Regulatory Requirements

- Compliance with institutional data protection policies to protect Student Personally Identifiable Information (PII).
- The OBE calculation engine must strictly adhere to the specific mapping and attainment logic defined by accreditation bodies.
- Implementation of legally defensible plagiarism to support disciplinary actions if necessary.

Technology Constraints

- Django Framework
- Use of open-source technologies only
- MySQL
-

Resource Constraints

Project execution is limited by academic timeline and infrastructure availability.

- Development timeline: 2 months (completion by April 2026)
- Development team: 4
- Hosting environment

These limitations require efficient planning, modular design, and realistic scope management.

11.Assumptions and Dependencies

Assumptions

The design and deployment of Evalify are based on the following assumptions:

- Users (students, faculty, and administrators) are assumed to have consistent internet connectivity to interact with the web-based OBE portal.

- It is assumed that all users possess and will use valid university-issued email addresses for registration and secure authentication.
- The system assumes that faculty members provide accurate and properly structured mappings between CLOs and PLOs.
- The hosting environment or university data center is assumed to maintain high uptime and adequate resource allocation for database operations.
- It is assumed that the raw marks entered by faculty are accurate, as these form the mathematical basis for all automated OBE attainment calculations.

Dependencies

The Evalify system depends on the following software frameworks, tools, and services:

- **Django Framework (Python Backend):**

  Used for implementing the MVC architecture, managing the OBE Calculation Engine, and handling ORM-based database interactions.

- **MySQL Database:**

  Used as the primary relational database to maintain complex relationships between Users, Courses, CLOs,PLOs and AttainmentRecords.

- **Django Authentication & RBAC:**

  Provides the secure framework for password hashing, session management, and enforcing Role-Based Access Control.

- **Third-Party Integrity APIs:** Required for the Integrity & Compliance module to perform real-time plagiarism checks .
- **Bootstrap & Frontend Technologies:** HTML5, CSS3, and JavaScript are used to build the responsive dashboards and visual attainment graphs.
- **Cloud Storage Infrastructure :** Necessary for the secure, scalable storage of student Submission files and faculty Academic_Resources.
- **Git & GitHub:**

  Utilized for version control, collaborative development, and maintaining the system's codebase integrity.

12. Traceability Matrix

12.1 Functional Requirements Traceability

Req ID	Requirement Description	System Module	Stakeholder
FR 1.1	Secure login/logout	Auth & RBAC	Admin, Faculty, Student
FR 1.2	Role-Based Access Control (RBAC)	Auth & RBAC	Admin, Faculty, Student
FR 2.1	Map assessment questions to CLOs and PLOs	OBE Mapping Engine	Faculty
FR 2.2	Tag questions with Bloom's Taxonomy levels	OBE Mapping Engine	Faculty
FR 2.3	Define custom weightage for CLOs per assignment	OBE Mapping Engine	Faculty
FR 3.1	Auto-calculate attainment upon mark entry	Assessment & Rubric	Faculty
FR 3.2	Generate individual and class-level reports	Resource & Analytics Module	Faculty
FR 3.3	Rubric-based grading interface	Resource & Analytics Module	Faculty

FR 4.1	Integrated plagiarism checker for submissions	Integrity & Plagiarism	Faculty, Student
FR 4.2	Support for multiple file formats	Resource & Analytics Module	Student
FR 5.1	Student progress navigator with visual charts	Resource & Analytics Module	Student
FR 5.2	Categorized upload of digital lecture notes/videos	Resource & Analytics Module	Faculty
FR 5.3	Repository for previous semester exam papers	Resource & Analytics Module	Student
FR 5.4	Institutional dashboard for aggregated PLO tracking	Resource & Analytics Module	Admin

12.2 Non-Functional Requirements Traceability

Req ID	Requirement Description	System Module	Stakeholder
NFR 1.1	System must support at least 500 concurrent users	Infrastructure	Admin, Students
NFR 1.2	OBE attainment calculations must be completed in under 5 seconds.	OBE Calculation Engine	Faculty, Admin
NFR 2.1	Passwords must be hashed using PBKDF2.	Auth & RBAC	All Users
NFR 2.2	Role-Based Access Control (RBAC) .	Auth & RBAC	Admin, Faculty

NFR 2.3	All data in transit	Communication Interface	All Users
NFR 3.1	99.9% system uptime.	Hosting	Admin, Students
NFR 3.2	Automated database backups every 24 hours	Database	Admin
NFR 4.1	Responsive UI	Frontend :Bootstrap	Students, Faculty
NFR 4.2	Intuitive dashboard for visualizing CLO/PLO attainment.	Resource & Analytics	Faculty, Students
NFR 5.1	Ability to add new departments or courses without code modification.	System Architecture	Admin
NFR 5.2	Modular code structure to support future integration of AI tools.	Backend (Django)	Developers

14. Appendices

Appendix A – Glossary

Term	Definition
ACID	Atomicity, Consistency, Isolation, Durability; a set of properties that guarantee database transactions are processed reliably.
API	Application Programming Interface; a set of protocols and tools for building software and applications.
Attainment	The measurement of the extent to which a student has achieved specific learning goals.

BAETE	Board of Accreditation for Engineering and Technical Education; the body responsible for accrediting engineering programs in Bangladesh.
Bloom's Taxonomy	A hierarchical model used to classify educational learning objectives into levels of complexity and specificity.
CLO	Course Learning Outcome; specific statements describing what students should know or be able to do by the end of a course.
CSRF	Cross-Site Request Forgery; a type of malicious exploit where unauthorized commands are submitted from a user the web application trusts.
Django	A high-level Python web framework that encourages rapid development and clean, pragmatic design.
LMS	Learning Management System; a software application for the administration, documentation, tracking, and delivery of educational courses.
MVC	Model-View-Controller; a software architectural pattern for implementing user interfaces on computers.
OBE	Outcome-Based Education; an educational theory that bases each part of an educational system around goals (outcomes).
ORM	Object-Relational Mapping; a technique that lets you query and manipulate data from a database using an object-oriented paradigm.
PLO	Program Learning Outcome; broad statements that describe what graduates of a program are expected to know and be able to do at the time of graduation.
RBAC	Role-Based Access Control; a method of restricting network access based on the roles of individual users within an enterprise.
SRS	Software Requirements Specification; a document that describes what the software will do and how it will be expected to perform.

TLS	Transport Layer Security ; cryptographic protocols designed to provide communications security over a computer network.

Appendix B

Appendix B contains the complete diagrams schema for ”Evalify”, including:

- ERD
- DFD
- Problem Flows
- Use Case Diagram
- Class Diagram

Appendix C – Supporting Documents

- Core Project Specification: The master document containing the full evolution, technical roadmap, and system vision for Evalify.
- Stakeholder Validation Data: Automated OBE tracking and real-time academic analytics.
- Technical Blueprints: ERDs, and Data Flow Diagrams (DFDs) detailing the integration of the Django backend with cloud storage.

—----------Thank You—----------